Happy Chhoti Holi (Holika Dahan) To Everyone.

Burnt My Inconsistency

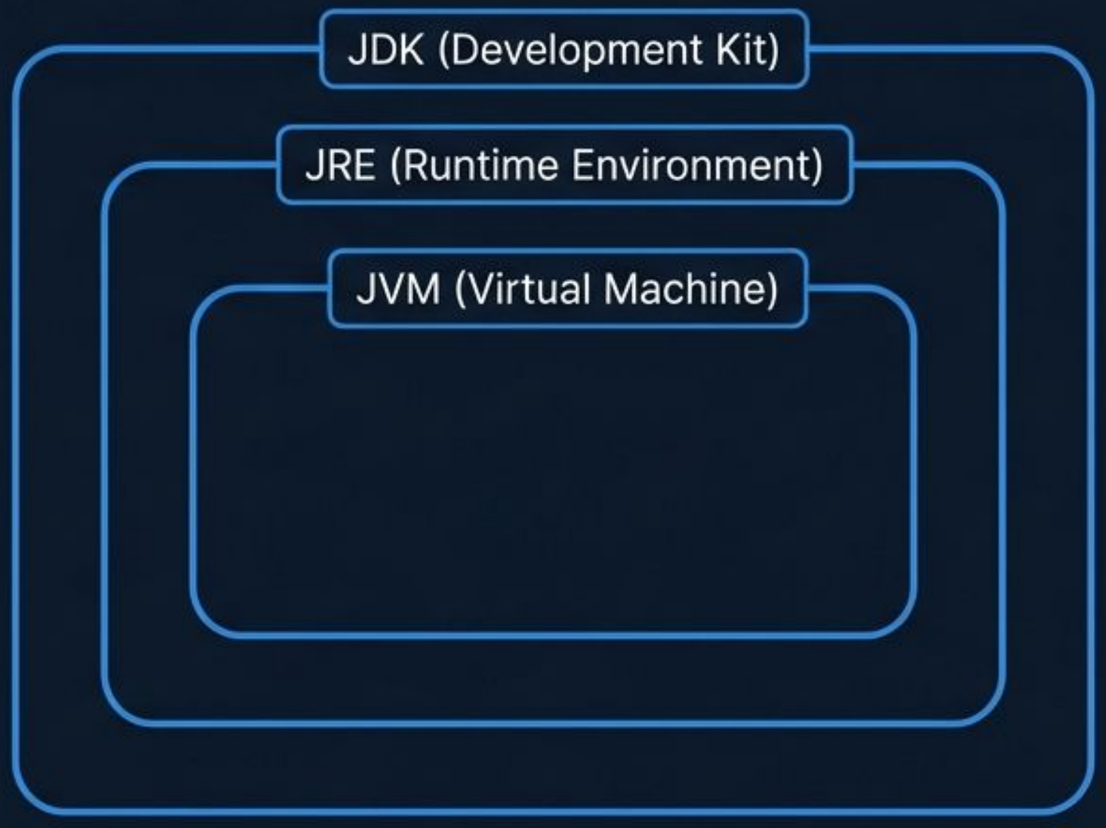
Day 2 of Consistency 🔥

☕ Day 2 — Java Series

JDK, JRE, JVM — Demystified

THE THREE PILLARS OF JAVA ARCHITECTURE

- The Hierarchy
  Java's architecture relies on three nested components.
- The 'Russian Doll' Model
  They are not separate entities; they exist inside one another.
- JDK (Outer)
  The full kit for developing applications.
- JRE (Middle)
  The environment required to run applications.
- JVM (Inner)
  The engine that actually executes the code.

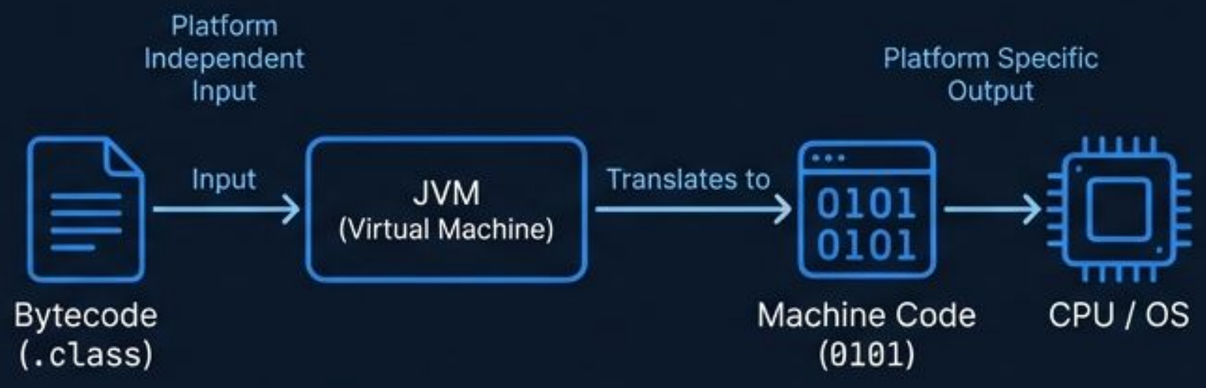


☕ Day 2 — Java Series

JVM — The Core Engine

JAVA VIRTUAL MACHINE

- **Virtual Computer**
  Software that simulates a physical machine within your RAM.
- **The Translator**
  Converts Bytecode (`.class`) into Machine Code (0s & 1s).
- **Platform Specific**
  JVM is unique for every OS (Windows, Mac, Linux).
- **Role**
  It bridges the gap: 'Write Once, Run Anywhere'.



☕ Day 2 — Java Series

JRE — The Runtime Home

JAVA RUNTIME ENVIRONMENT

- **Purpose**
  The minimum environment required to run Java apps.
- **The Equation**
  JRE = JVM + Class Libraries.
- **Libraries**
  Contains pre-written helper code (e.g., math, io).
- **Dependency**
  Without these, JVM cannot execute commands like printing.

☕ Day 2 — Java Series

JDK — The Developer's Toolkit

JAVA DEVELOPMENT KIT

- **Target Audience**

  Required only for developers writing and compiling code.

- **The Equation**

  JDK = JRE + Development Tools.

- **Key Tools**

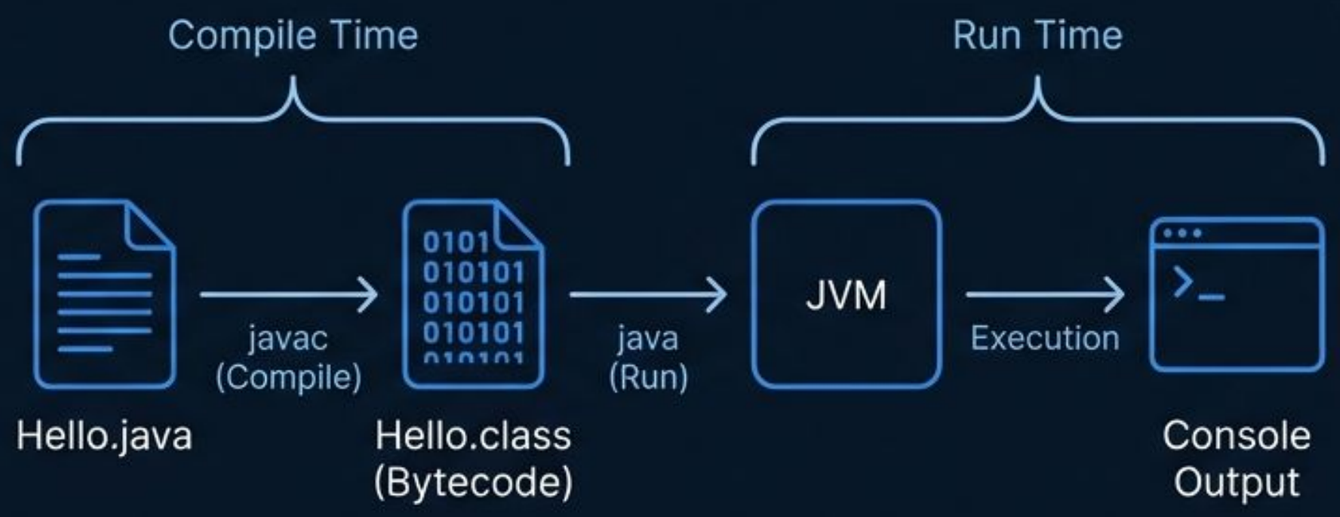
  `javac`: The Compiler (Source → Bytecode).
  `debugger`: For inspecting and fixing code execution.
  `javadoc`: For generating documentation.

☕ Day 2 — Java Series

How Your Code Runs

COMPILATION FLOW STEP BY STEP

- **1. Source Code**
  Human-readable text file (Hello.java).
- **2. Compilation**
  The 'javac' command converts source into Bytecode.
- **3. Execution**
  The 'java' command triggers the JVM.
- **The Output**
  JVM translates bytecode to machine code for hardware.

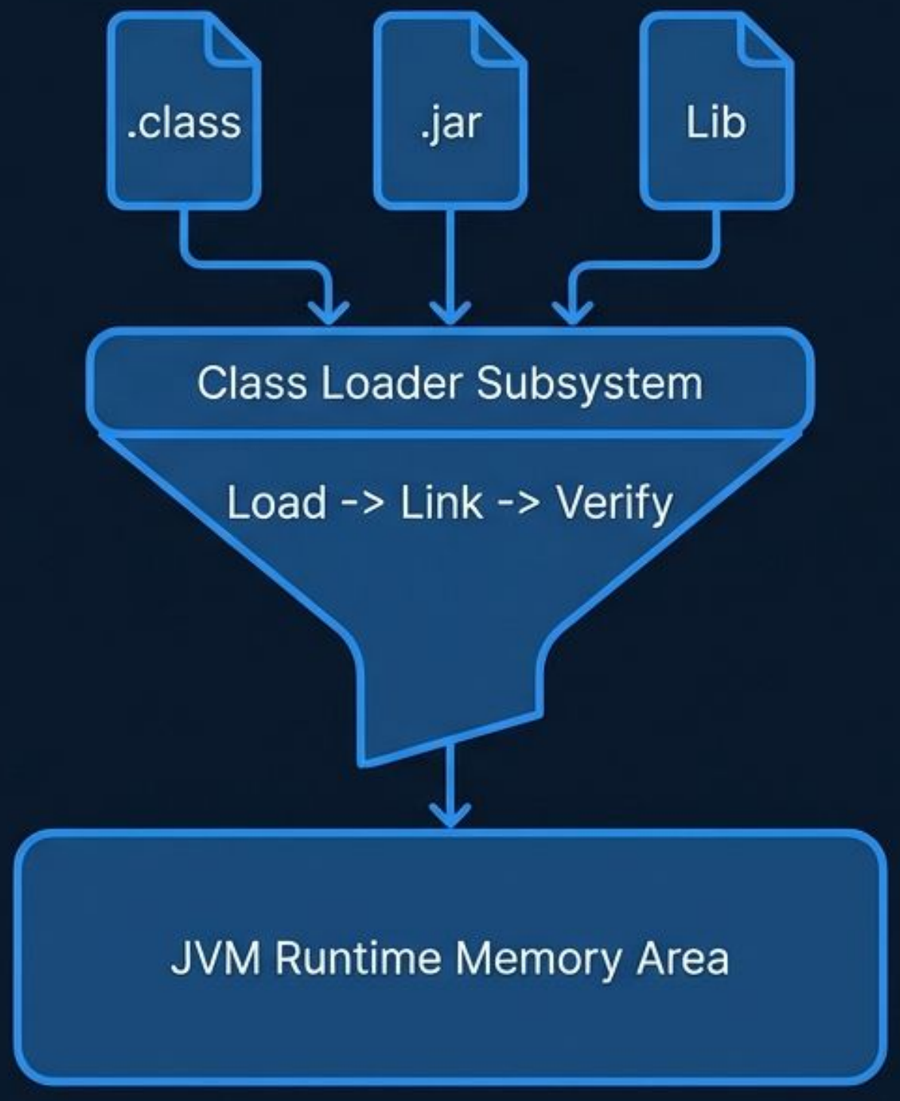


☕ Day 2 — Java Series

JVM Internals — Class Loading

THE ENTRY POINT

- **Lazy Loading**
  JVM doesn't load everything at start; loads on demand.
- **Linking**
  Connects your code with standard JRE libraries.
- **Verification**
  Checks bytecode for security violations before execution.
- **Role**
  The gatekeeper that ensures code is safe and complete.

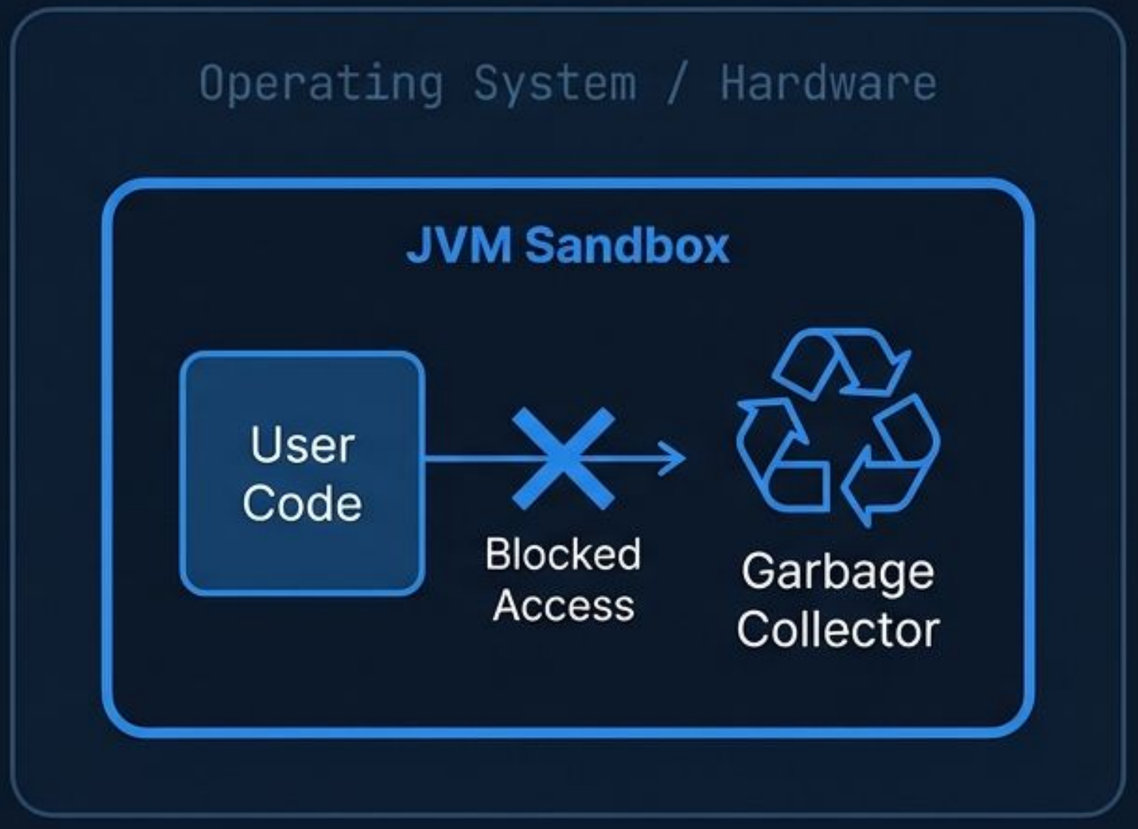


☕ Day 2 — Java Series

JVM Security & Memory

THE SANDBOX MODEL

- The Sandbox
  Java code runs inside the JVM, restricted from OS access.
- Security
  Prevents malicious code from deleting files or accessing kernel.
- Garbage Collection
  Automatic memory management. JVM identifies and deletes unused objects to free RAM.

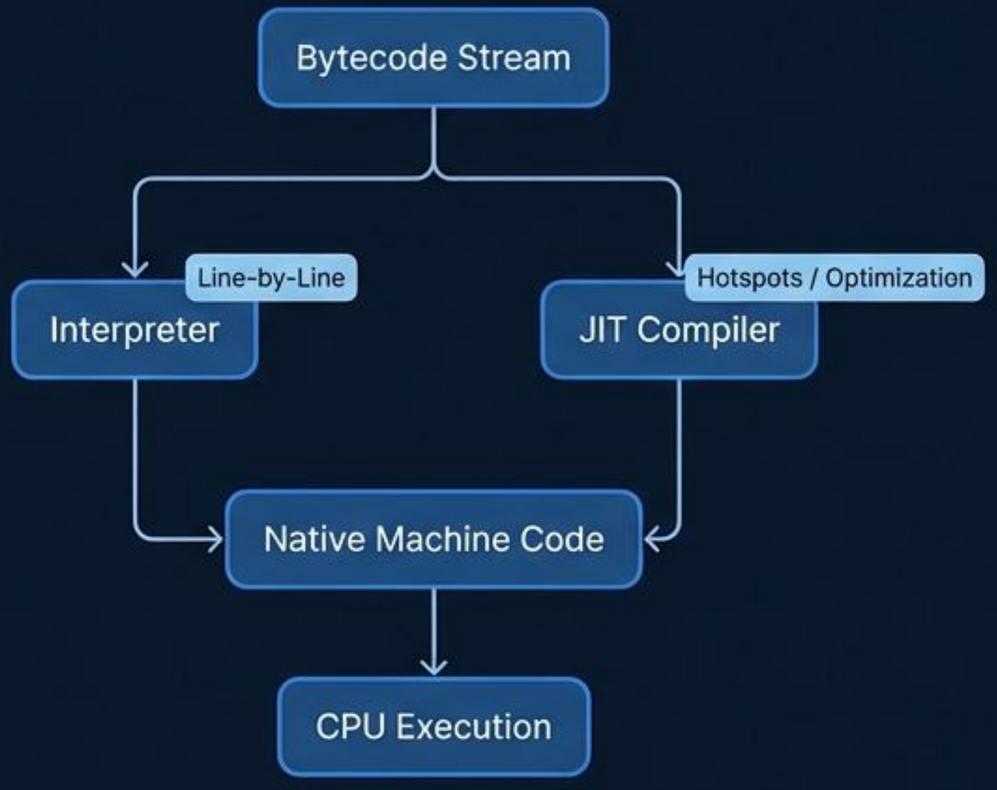


☕ Day 2 — Java Series

Execution Engine

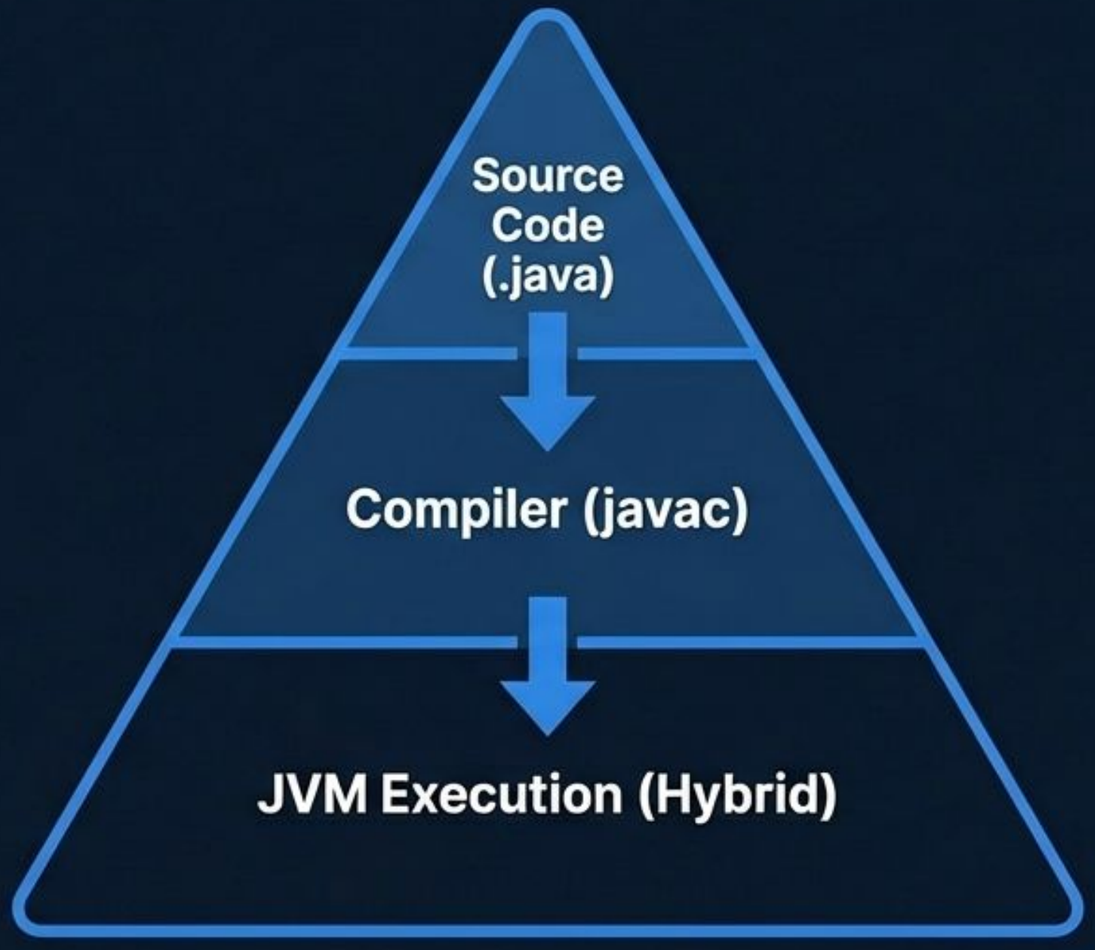
INTERPRETER VS JIT COMPILER

- **Hybrid Model**
  Java is both Compiled (to bytecode) and Interpreted.
- **Interpreter**
  Executes bytecode line-by-line. Fast startup, slower run.
- **JIT (Just-In-Time)**
  Identifies 'Hotspots' (frequent code).
  Compiles them directly to native machine code.
- **Result**
  Fast startup time + High performance over time.



☕ Day 2 — Java Series

Day 2 Reflection

KEY TAKEAWAYS & EDITIONS

- **Architecture**
  JDK builds it, JRE provides libraries, JVM runs it.
- **Performance**
  Hybrid Engine (Interpreter + JIT) balances speed & portability.
- **Java Editions**
  JSE (Standard): Core Java (What we are learning).
  JEE (Enterprise): For Web/Large systems.
  JME (Micro): Legacy mobile (obsolete).